

Working With ad hoc Commands & Ansible Modules:

- An **ad hoc command** is a way of executing a single Ansible task quickly, one that you do not need to save to run again later.
- They are simple, online operations that can be run without writing a playbook. Ad hoc commands are useful for quick tests and changes. For example, you can use an ad hoc command to make sure that a certain line exists in the **/etc/hosts** file on a group of servers. You could use another ad hoc command to efficiently restart a service on many different machines, or to ensure that a particular software package is up-to-date.
- **Ad hoc** commands are very useful for quickly performing simple tasks with Ansible.
- In many situations, however, **ad hoc** commands are exactly the tool you need to perform simple tasks quickly.
- Running **Ad Hoc** Commands Use the **ansible** command to run ad hoc commands:
`ansible host-pattern -m module [-a 'module arguments'] [-i inventory]`
- The **host-pattern** argument is used to specify the managed hosts on which the **ad hoc** command should be run. It could be a specific managed host or host group in the inventory.
- The **-m** option takes as an argument the name of the module that Ansible should run on the targeted hosts.
- **Modules** are small programs that are executed to implement your task. Some modules need no additional information, but others need additional arguments to specify the details of their operation. The **-a** option takes a list of those arguments as a quoted string.
- One of the simplest ad hoc commands uses the **ping** module. This module does not do an ICMP ping, but checks to see if you can run Python-based modules on managed hosts.
- **Modules** are the tools that **ad hoc** commands use to accomplish tasks. Ansible provides hundreds of modules which do different things. You can usually find a tested, special-purpose module that does what you need as part of the standard installation.
- The `ansible-doc -l` command lists all modules installed on a system.
- You can use `ansible-doc` to view the documentation of particular modules by name, and find information about what arguments the modules take as options. For example, the following command displays documentation for the ping module:

```
[root@myanisblevc ansibleu]# ansible-doc ping
```

Some Important Modules that you should Learn:

Module Category	Module	Description
Files Modules	copy	Copy a local file to the managed host
	file	Set permissions and other file properties
	lineinfile	Ensure a specific line exists or is absent in a file
	synchronize	Synchronize content using rsync
Software Package Modules	package	Manage packages using the OS-native package manager
	yum	Manage packages using YUM
	apt	Manage packages using APT
	dnf	Manage packages using DNF
	gem	Manage Ruby gems
	pip	Manage Python packages from PyPI
System Modules	firewalld	Manage ports and services using firewalld
	reboot	Reboot a machine
	service	Manage system services
	user	Add, remove, and manage user accounts
Net Tools Modules	get_url	Download files over HTTP, HTTPS, or FTP
	nmcli	Manage networking using nmcli
	uri	Interact with web services and REST APIs

- Most modules take arguments. You can find the list of arguments available for a module in the module's documentation. Ad hoc commands pass arguments to modules using the **-a** option. When no argument is needed, omit the **-a** option from the ad hoc command. If multiple arguments need to be specified, supply them as a quoted space-separated list.

```
[root@myanislebvc ansible]# ansible all -m user -a 'name=newbie uid=4000 state=present'
```

- Most modules are **idempotent**, which means that they can be run safely multiple times, and if the system is already in the correct state, they do nothing.

Running Arbitrary Commands on Managed Hosts

Command,shell,raw modules:

- The **command** module allows administrators to run arbitrary commands on the command line of managed hosts.
- The **command** to be run is specified as an argument to the module using the **-a** option.

```
[root@myanislebvc ansible]# ansible mymanagedhosts -m command -a /usr/bin/hostname
```

- The **command** module allows administrators to quickly execute remote commands on managed hosts.
- These **commands** are not processed by the **shell** on the managed hosts. As such, they cannot access shell environment variables or perform shell operations, such as redirection and piping.
- For situations where commands require **shell** processing, administrators can use the **shell module**. Like the **command** module, you pass the commands to be executed as arguments to the module in an ad hoc command. Ansible then executes the command remotely on the managed hosts. Unlike the command module, the commands are processed through a **shell** on the managed hosts. Therefore, **shell** environment variables are accessible and shell operations such as redirection and piping are also available for use.

EXAMPLE

```
[root@myanislebvc ansible]# ansible localhost -m command -a set
```

```
[root@myanislebvc ansible]# ansible localhost -m shell -a set
```

- Both **command and shell** modules require a working Python installation on the managed host.
- A third module, **raw**, can run commands directly using the remote shell, bypassing the module subsystem.
- This is useful when managing systems that cannot have Python installed (for example, a network router). It can also be used to install Python on a host.

Example:

```
[root@myanisblevc ansibleu]# ansible kali -m raw -a "cat /etc/os-release"
```

```
192.168.244.194 | CHANGED | rc=0 >>
```

```
PRETTY_NAME="Kali GNU/Linux Rolling"
```

```
NAME="Kali GNU/Linux"
```

```
VERSION_ID="2025.1"
```

```
VERSION="2025.1"
```

```
VERSION_CODENAME=kali-rolling
```

```
ID=kali
```

```
ID_LIKE=debian
```

```
HOME_URL="https://www.kali.org/"
```

```
SUPPORT_URL="https://forums.kali.org/"
```

```
BUG_REPORT_URL="https://bugs.kali.org/"
```

```
ANSI_COLOR="1;31"
```

```
Shared connection to 192.168.244.194 closed.
```

```
[root@myanisblevc ansibleu]# ansible kali -m command -a "cat /etc/os-release"
```

```
192.168.244.194 | CHANGED | rc=0 >>
```

```
PRETTY_NAME="Kali GNU/Linux Rolling"
```

```
NAME="Kali GNU/Linux"
```

```
VERSION_ID="2025.1"
```

VERSION="2025.1"

VERSION_CODENAME=kali-rolling

ID=kali

ID_LIKE=debian

HOME_URL="https://www.kali.org/"

SUPPORT_URL="https://forums.kali.org/"

BUG_REPORT_URL="https://bugs.kali.org/"

ANSI_COLOR="1;31"

[root@myanislebvc ansibleu]#

- See output of both modules, in raw module output you will see

Shared connection to 192.168.244.194 closed

Whereas its not in output of "command module", because

Ansible sends the command directly over SSH, without using Python, without using the Ansible execution environment, and without any persistent connection wrapper. Because of that, the SSH client prints its own message,

it simply means:

- SSH opened a connection
- Sent the raw command
- Received output
- Closed the connection

Note:

*In most circumstances, it is a recommended practice that you avoid the **command, shell, and raw "run command" modules**. Most other modules are idempotent and can perform change tracking automatically. They can test the state of systems and do nothing if those systems are already in the correct state. By contrast, it is much more complicated to use "run command" modules in a way that is idempotent. Depending upon them makes it harder for you to be confident that rerunning an ad hoc command or playbook would not cause an unexpected failure. When a **shell or command module runs**, it typically reports a **CHANGED** status based on whether it thinks it affected machine state. There are times when "run command" modules are valuable tools and a good solution to a problem. If you do need to use them, it is probably best to try to use the **command** module first, resorting to **shell** or **raw** modules only if you need their special feature*

CONFIGURING CONNECTIONS FOR AD HOC COMMANDS

- The directives for managed host connections and privilege escalation can be configured in the Ansible configuration file, and they can also be defined using options in **ad hoc commands**.
- When defined using options in ad hoc commands, they take precedence over the directive configured in the Ansible configuration file.
- The following table shows the analogous command-line options for each configuration file directive.

Ansible Command-Line Options

Configuration File Directive	Command-Line Option	Description
inventory	-i	Specify inventory file or directory
remote_user	-u	SSH user to connect as
become	--become, -b	Enable privilege escalation
become_method	--become-method	Set privilege escalation method (sudo, su, pbrun, etc.)
become_user	--become-user	User to become after privilege escalation
become_ask_pass	--ask-become-pass, -K	Prompt for privilege escalation password

